

[Deprecated] Примеры тестов на Python (unittest)

unittest - старый способ запуска тестов на Python, использует стандартную библиотеку unittest.

Указать используемый вариант можно в файле test.entry, использовав ключевое слово **py_unittest**.

Пример простейшего test.entry для unittest:

```
1 {  
2     "py_unittest": "test_cloud"  
3 }
```

Измерение покрытия кода в тестах на Python

В [SBIS SDK](#) версии 23.7000 был добавлен Python-фреймворк [coverage.py](#), который позволяет измерять покрытие кода в питоновских файлах. Он работает с обеими библиотеками pytest и unittest.

Он добавлен в SBIS SDK как самостоятельный python-пакет и лежит в директории tools/python_test_utility. Его можно запустить как самостоятельную программу вместо интерпретатора Python из каталога **{SBIS SDK}/tools/python_test_utility/bin/coverage**.

В тестовый фреймворк добавлена интеграция с coverage.py. Для ее использования в тестовый фреймворк добавлены два параметра:



Параметры необходимо передавать в строку запуска test_framework.

- `--py_coverage` - при передаче активирует сбор информации о покрытии кода;
- `--py_coverage_format FORMAT` - указать формат отчета (html, xml, json). По умолчанию html.

В test.entry введен параметр coverage_args в него можно передать параметры конструктора класса [Coverage](#) в формате json, например:

```
1 "coverage_args" : {  
2     "include" : "my_source_dir_name",  
3     "concurrency" : "thread"  
4 }
```

При его наличии он заменяет параметры, равные по умолчанию:

```
1 {  
2     "omit" : [  
3         "*/test_framework/*",  
4         "*/python_test_utility/*",  
5         "*/python_library/*",  
6         f"*/{test_dir}/*"  
7     ],  
8     "concurrency" : "thread"  
9 }
```

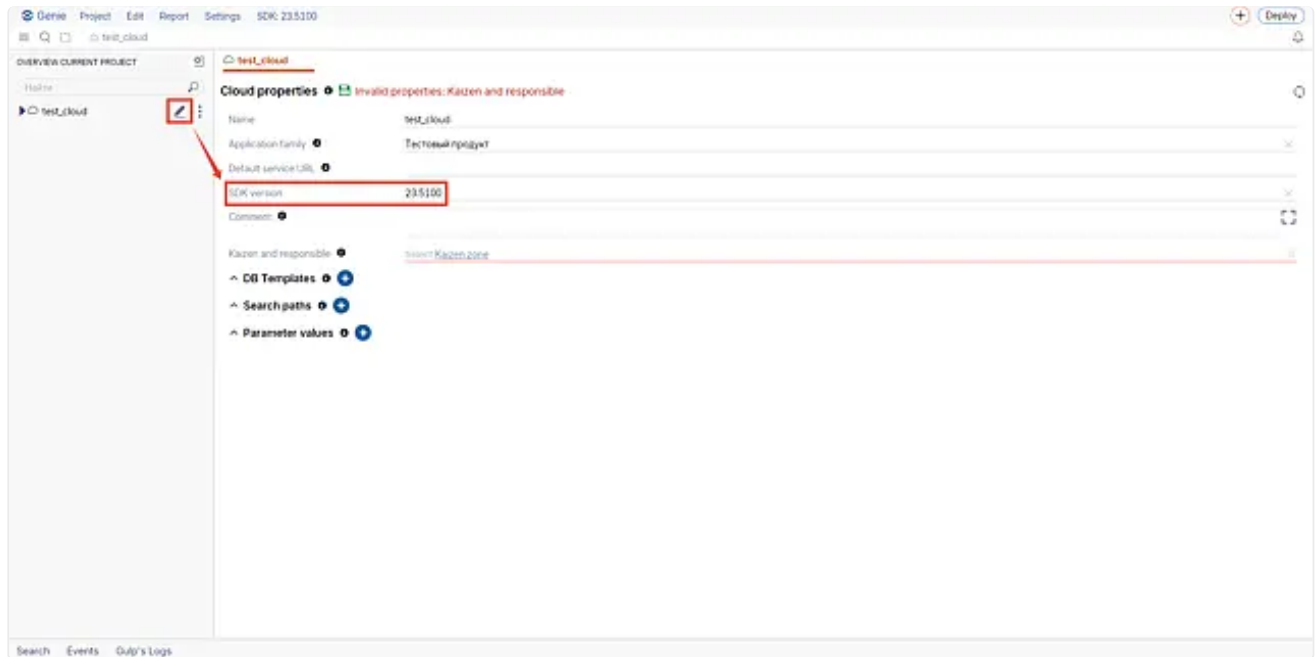


Отчет о покрытии кода по умолчанию генерируется в рабочую директорию теста в папку coverage_{test_name}, либо в папку, указанную в параметре `--xunit_output`.

Подготовка к запуску тестов

Все примеры тестов выполнены в SBIS SDK версии 23.5100. Если вы установили SBIS SDK другой версии, поменяйте ее номер в файле CMakeLists.txt.

Откройте s3cld-файл проекта в [Genie](#) и в свойствах проверьте номер версии SBIS SDK.



Для запуска тестов в docker-контейнере активируйте виртуальное окружение с помощью команды:

```
1 source /venv_conan/bin/activate (папка в корне)
```

Порядок тестирования и набор тестов аналогичны [примерам тестов на C++](#) — там же можно посмотреть более подробное описание тестов.

При разработке тестов для модулей БЛ скорее всего возникнет проблема "жирного облака", когда тестируемый модуль зависит от множества других напрямую или косвенно. В этом случае рекомендуется часть модулей заменить своими заглушками (пустой модуль с таким же uuid), часть функционала сторонних модулей перекрыть своими реализациями или мок-сервисом (когда нужно перекрыть именно удаленный вызов).

Запуск тестов

Для запуска теста выполните следующие шаги:

1. Создайте каталог **build** для сборки тестовых проектов. Размещение этого каталога определяется разработчиком. В нашем примере каталог создается на одном уровне с **my_project**.

```
├─ build
├─ my_project
│   └─ tests
│       └─ simple_test_suite
│           ├── clouds
│           │   └─ test_cloud
│           │       ├── test_cloud.s3cld
│           │       └─ test.s3srv
│           ├── src
│           │   ├── CMakeLists.txt
│           │   └─ simple_test.py
│           └─ test.entry
```

2. Запустите тест:

- Откройте командную строку/консоль:
 - При работе в Windows откройте [Tensor C++ Developer Command Prompt 2025](#).
 - При работе в Linux откройте консоль и с помощью команды export [добавьте переменные окружения](#).
- Перейдите в каталог **build**.
- Запустите [CMake](#) для генерации проектных файлов
 - Запуск генерации проекта.

- Windows 32bit:

```
1 cmake ..\my_project\tests\<имя тестового проекта>\src ^
2 -DSBIS_UNIT_TESTS=1 -G Ninja ^
3 -DCMAKE_TOOLCHAIN_FILE="%TensorCppDev2%\windows-toolchain-
  i686.cmake"
```

- Windows 64bit:

```
1 cmake ..\my_project\tests\<имя тестового проекта>\src ^
2 -DSBIS_UNIT_TESTS=1 -G Ninja ^
3 -DSBIS_ARCHITECTURE=x86_64
```

- Linux и macOS на Intel (x86_64):

```
1 cmake ../my_project/tests/<имя тестового проекта>/src \
2 -DSBIS_UNIT_TESTS=1 -G Ninja
```

- macOS на Apple M1 (ARM):

```
1 cmake ../my_project/tests/<имя тестового проекта>/src \
2 -DSBIS_ARCHITECTURE=arm64 \
3 -DSBIS_UNIT_TESTS=1 -G Ninja
```

3. Запуск сборки проекта.

ninja

4. Запуск тестирования проекта. Команды написаны для SBIS SDK 23.5100.

- Windows 32bit:

```
1 py -3.11-32 "%SBISPlatformSDK_235100%\tools\test_framework"
  "..\my_project\tests" ^
2 --jinnee "%SBISPlatformSDK_235100%\tools\jinnee"
```

- Windows 64bit:

```
1 py -3.11-64 "%SBISPlatformSDK_235100%\tools\test_framework"
  "..\my_project\tests" ^
2 --jinnee "%SBISPlatformSDK_235100%\tools\jinnee"
```

- Linux:

```
1 /opt/python3.11/bin/python3.11
  $SBISPlatformSDK_235100/tools/test_framework ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```

- macOS:

```
1 python3.11 $SBISPlatformSDK_235100/tools/test_framework
  ../my_project/tests \
2 --jinnee $SBISPlatformSDK_235100/tools/jinnee
```

```
C:\Program Files (x86)\TensorCppDev-2022>cd C:\python\my_project01\build
C:\python\my_project01\build>cmake ..\my_project\tests\simple_test_suite\src ^
Продолжить? -DSBIS_UNIT_TESTS=1 -G Ninja ^
Продолжить? -DCMAKE_TOOLCHAIN_FILE="C:\TensorCppDev2\windows-toolchain-1606.cmake"
CMake Warning (dev) in CMakeLists.txt:
  No project() command is present. The top-level CMakeLists.txt file must
  contain a literal, direct call to the project() command. Add a line of
  code such as

  project(ProjectName)

  near the top of the file, but after cmake_minimum_required().

  CMake is pretending there is a "project(Project)" command on the first
  line.
  This warning is for project developers. Use -Wno-dev to suppress it.
-- Generating done (0.0s)
-- Build files have been written to: C:\python\my_project01\build

C:\python\my_project01\build>ninja
[2/2] Linking CXX executable bin/test-some-lib.exe

C:\python\my_project01\build>py -3.11-32 "C:\SBISPlatformSDK_235100\tools\test_framework" "..\my_project\tests" ^
Продолжить? --jlnnee "C:\SBISPlatformSDK_235100\tools\jlnnee"
[1] Running test 'simple_test_suite' (171) ...
Finding zombie processes in 'C:\python\my_project01\build\tests\simple_test_suite\test'...
No zombies found
[1] Deploying service C:\python\my_project01\build\tests\simple_test_suite from C:\python\my_project01\my_project\tests\simple_test_suite\clouds\test_cloud\test_cloud.s3cld
Finding zombie processes in 'C:\python\my_project01\build\tests\simple_test_suite\setup_before'...
No zombies found
Finding zombie processes in 'C:\python\my_project01\build\tests\simple_test_suite\setup'...
No zombies found
[1] *** SUCCESS
*** No errors detected
```

Простой тест



Файлы для запуска теста

Архив с файлами для запуска теста можно скачать [здесь](#).

Файловая структура каталога теста имеет следующий вид:

```
├─ my_project
│   └─ tests
│       └─ simple_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   └─ test_cloud // каталог тестового приложения
│           │       ├── test_cloud.s3cld // тестовое облако
│           │       └─ test.s3srv // тестовый сервис
│           ├── src // каталог с исходниками тестов
│           │   ├── CMakeLists.txt // файл сборки теста
│           │   └─ simple_test.py // исходный код теста
│           └─ test.entry // файл описания теста
```

В каталоге **clouds** располагается каталог **test_cloud** с метаданными проекта - файлом облака **test_cloud.s3cld** и тестовым сервисом **test.s3srv**. Имя каталога должно совпадать с именем **s3cld**-файла.

В каталоге **src** расположены файл **simple_test.py**, в котором размещен исходный код теста.

```
1 import sbis_root as sbis
2 import os
3 from unittest import TestCase
4
5 class TextFrameworkExample(TestCase):
6
7     def test_simple(self):
8         self.assertEqual(1, 1)
```

и файл сборки теста **CMakeLists.txt**:

```
1 cmake_minimum_required(VERSION 3.5.2)
2 include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3 sbis_create_project("test-some-lib" test DEPENDS
4                     boost_system
5                     boost_filesystem
6                     boost_unit_test_framework)
```

```
7 sbis_add_sdk_dependency( sbis-lib300 )
```

Файл test.entry будет выглядеть следующим образом:

```
1 {
2     "pytest_on": "test_cloud"
3 }
```

Простой тест с использованием ресурсов



Файлы для запуска теста

Архив с файлами для запуска теста можно скачать [здесь](#).

Файловая структура каталога теста имеет следующий вид:

```
├─ my_project
│   └─ tests
│       └─ res_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   ├── test_cloud // каталог тестового приложения
│           │   │   ├── test_cloud.s3cld // тестовое облако
│           │   │   └─ test.s3srv // тестовый сервис
│           ├── resources // каталог для дополнительных файлов
│           │   ├── hello_world.txt // текстовый файл
│           ├── src // каталог с исходниками тестов
│           │   ├── CMakeLists.txt // файл сборки теста
│           │   └─ resources_test.py // исходный код теста
│           └─ test.entry // файл описания теста
```

В каталоге **clouds** располагаются метаданные проекта - файл облака test_cloud.s3cld и тестовый сервис test.s3srv.

Каталог **resources** необходим для теста с использованием ресурсов. В этот каталог помещаются дополнительные файлы, на которые после запуска теста будут созданы символические ссылки (в данном примере - файл hello_world.txt с текстом 'Hello, World').

В каталоге **src** расположен файл resources_test.py, в котором размещен исходный код теста

```
1 import sbis_root as sbis
2 import os
3 from unittest import TestCase
4
5 class TextFrameworkExample(TestCase):
6
7     def test_resources(self):
8         self.assertTrue( os.path.exists("hello_world.txt"))
9
10        with open("hello_world.txt", "r") as f:
11            readFile = f.read()
12            self.assertEqual(readFile, 'Hello, World?')
```

Файл test.entry будет выглядеть следующим образом:

```
1 {
2     "pytest_on": "test_cloud"
3 }
```

В результате тестирования, содержимое тестируемого файла не соответствует строке теста:

```

C:\python\my_project02\build>py -3.11-32 "XSBISPlatformSDK_235100%\tools\test_framework" "..\my_project\tests" ^
Продолжить? --jinnex "XSBISPlatformSDK_235100%\tools\jinnex"
[1] Running test 'res_test_suite' (1/1) ...
Finding zombie processes in 'C:\python\my_project02\build\tests\res_test_suite\test'...
No zombies found
[1] Deploying service C:\python\my_project02\build\tests\res_test_suite from C:\python\my_project02\my_project\tests\res_test_suite\clouds\test_cloud\test_cloud.s3cld
[1] *** ERRORS FOUND
test_resources (resources_test.TextFrameworkExample.test_resources)
Traceback (most recent call last):
  File "C:\python\my_project02\my_project\tests\res_test_suite\src\resources_test.py", line 14, in test_resources
    self.assertEqual(readFile, 'Hello, World?')
AssertionError: 'Hello, World!' != 'Hello, World?'
- Hello, World!
+
+ Hello, World?
+
.....

[1] *** FAIL
[1] Error while running the test [PyTester finished with non-zero code: [1]]
[-] Failed tests:
[-] res_test_suite
*** 1 failures detected

```

Замените в файле resources_test.py 'Hello, World?' на 'Hello, World!'. Теперь результат теста положительный.

```

C:\python\my_project02\build>py -3.11-32 "XSBISPlatformSDK_235100%\tools\test_framework" "..\my_project\tests" ^
Продолжить? --jinnex "XSBISPlatformSDK_235100%\tools\jinnex"
[1] Running test 'res_test_suite' (1/1) ...
Finding zombie processes in 'C:\python\my_project02\build\tests\res_test_suite\test'...
No zombies found
[1] Deploying service C:\python\my_project02\build\tests\res_test_suite from C:\python\my_project02\my_project\tests\res_test_suite\clouds\test_cloud\test_cloud.s3cld
Finding zombie processes in 'C:\python\my_project02\build\tests\res_test_suite\res_test_suite\setup_before'...
No zombies found
Finding zombie processes in 'C:\python\my_project02\build\tests\res_test_suite\setup'...
No zombies found
[1] *** SUCCESS
*** No errors detected

```

Тест с использованием Redis



Файлы для запуска теста

Архив с файлами для запуска теста можно скачать [здесь](#).

Файловая структура каталога теста имеет следующий вид:

```

├─ my_project
│   └─ tests
│       └─ redis_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   └─ test_cloud // каталог тестового приложения
│           │       ├── test_cloud.s3cld // тестовое облако
│           │       └─ test.s3srv // тестовый сервис
│           ├── src // каталог с исходниками тестов
│           │   ├── CMakeLists.txt // файл сборки теста
│           │   └─ redis_test.py // исходный код теста
│           └─ test.entry // файл описания теста

```

В каталоге **clouds** располагаются метаданные проекта - файл облака test_cloud.s3cld и тестовый сервис test.s3srv.

В каталоге **src** расположены файл simple_test.py, в котором размещен исходный код теста:

```

1 import sbis_root as sbis
2 import os
3 from unittest import TestCase
4
5 class TextFrameworkExample(TestCase):
6
7     def test_redis(self):
8         redisClient = sbis.RedisClient ( "Тест.Хранилище", True )
9         key = "123"
10        key_exists = redisClient.IsExists( key );
11        self.assertEqual( key_exists, False)
12
13        redisClient.SetValue(key, "Value")

```

```

14         value = redisClient.Get( key );
15         self.assertEqual(value, "Value")

```

и файл сборки теста CMakeLists.txt:

```

1 cmake_minimum_required(VERSION 3.5.2)
2 include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3 sbis_create_project("test-some-lib" test DEPENDS
4                     boost_system
5                     boost_filesystem
6                     boost_unit_test_framework)
7 sbis_add_sdk_dependency( sbis-lib300 )

```

Для того чтобы использовать Redis в юнит-тестах, укажите в файле test.entry имя хранилища Redis:

```

1 {
2     "pytest_on": "test_cloud",
3     "redis" : ["Тест.Хранилище"]
4 }

```

Тест с использованием mock-сервиса

Подробное описание теста с использованием mock-сервиса описано [здесь](#).

Тест с запуском сайта



Файлы для запуска теста

Архив с файлами для запуска теста можно скачать [здесь](#).

Подробное описание теста с запуском сайтов описано [здесь](#). Повторите этот тест на Python.

Если у вас есть структура каталога, где лежит файл test_cloud.s3cld:

```

├─ my_project
│   └─ tests
│       └─ site_test_suite
│           ├── clouds // каталог для метаданных проекта
│           │   ├── SomeModule // каталог тестового модуля
│           │   │   ├── SomeModule.orx // справочник объектов
│           │   │   └─ SomeModule.s3mod // тестовый модуль
│           │   └─ test_cloud // каталог тестового приложения
│           │       ├── test_cloud.s3cld // тестовое облако
│           │       └─ test.s3srv // тестовый сервис
│           ├── resources // каталог для дополнительных файлов
│           └─ sbis-rpc-service.ini // файл с параметрами тестового
облака
│   └─ src // каталог с исходниками тестов
│       ├── CMakeLists.txt // файл сборки теста
│       ├── service_test.py // исходный код теста
│       └─ test.entry // файл описания теста

```

а файл test.entry имеет следующий вид:

```

1 {
2     "services" : [
3         {

```

```

4         "cloud": "test_cloud"
5     }
6 ],
7
8     "pytest_on": "test_cloud"
9 }

```

то при развороте будет сформирована структура каталогов:

```

├─ build
│   └─ tests
│       └─ site_test_suite
│           └─ run
│               └─ sbis-rpc-service.ini
├─ my_project
│   └─ tests
│       └─ SiteTestSuite
│           ├── clouds
│           │   ├── SomeModule
│           │   │   ├── SomeModule.orx
│           │   │   └─ SomeModule.s3mod
│           │   └─ test_cloud
│           │       ├── test_cloud.s3cld
│           │       └─ test.s3srv
│           ├── resources
│           │   └─ sbis-rpc-service.ini
│           ├── src
│           │   ├── CMakeLists.txt
│           │   └─ service_test.py
│           └─ test.entry

```

и в файл sbis-rpc-service.ini каталога **run** будут добавлены секции:

```

1  [test]
2  Адрес = http://r02-sizonenko1.region.tensor.ru:10000
3  БазаДанных = postgresql: host='localhost'
4                      port='5432'
5                      dbname='_sbis_test_db_2023-04-02T22-14-13-
6                      062939'
7                      user='postgres'
                          password='<пароль вашей локальной БД>'

```

Откройте проект test_cloud из каталога **clouds** с помощью [Genie](#). В модуле SomeModule в объекте MyObject добавлен метод AnotherMethod:


```

├── SomeModule.dpack // пакет данных
├── SomeModule.s3mod // файл тестового модуля
├── test_cloud // каталог тестового приложения
│   ├── test_cloud.s3cld // файл тестового облака
│   └── test.s3srv // тестовый сервис
├── resources // каталог для дополнительных файлов
│   └── sbis-rpc-service.ini // файл с параметрами тестового
облака
├── src // каталог с исходниками тестов
│   ├── CMakeLists.txt // файл сборки теста
│   └── service_test.py // исходный код
└── test.entry // файл описания теста

```

Файл тестового облака test_cloud открывается в [Genie](#). В настройках проекта в параметре **DB Templates** создана схема развертывания БД test_db.

В облаке проекта test_cloud создается тестовый модуль SomeModule.s3mod, который содержит:

- [Словарь данных](#) SomeModule с таблицей ТестоваяТаблица.

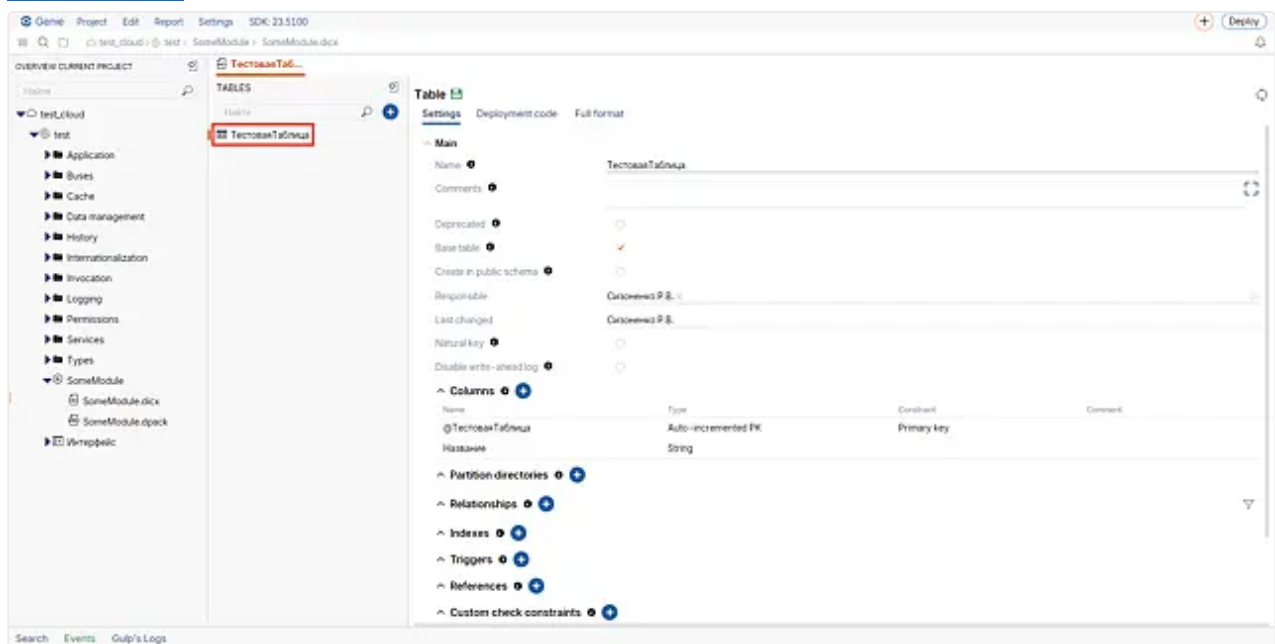
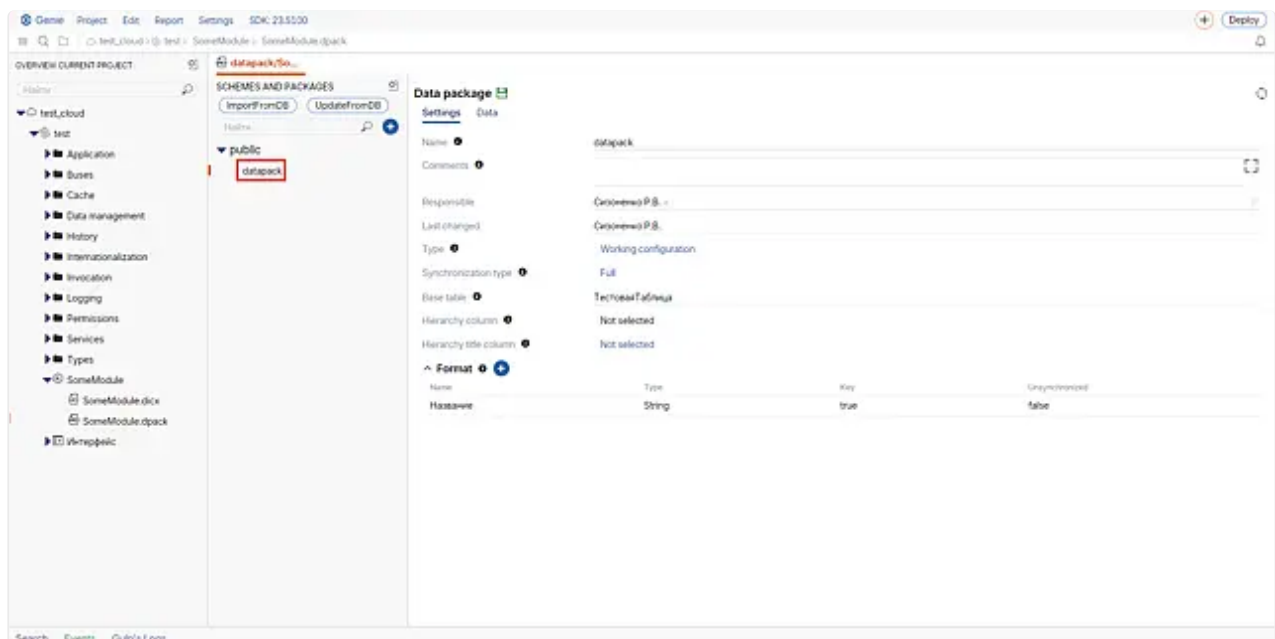
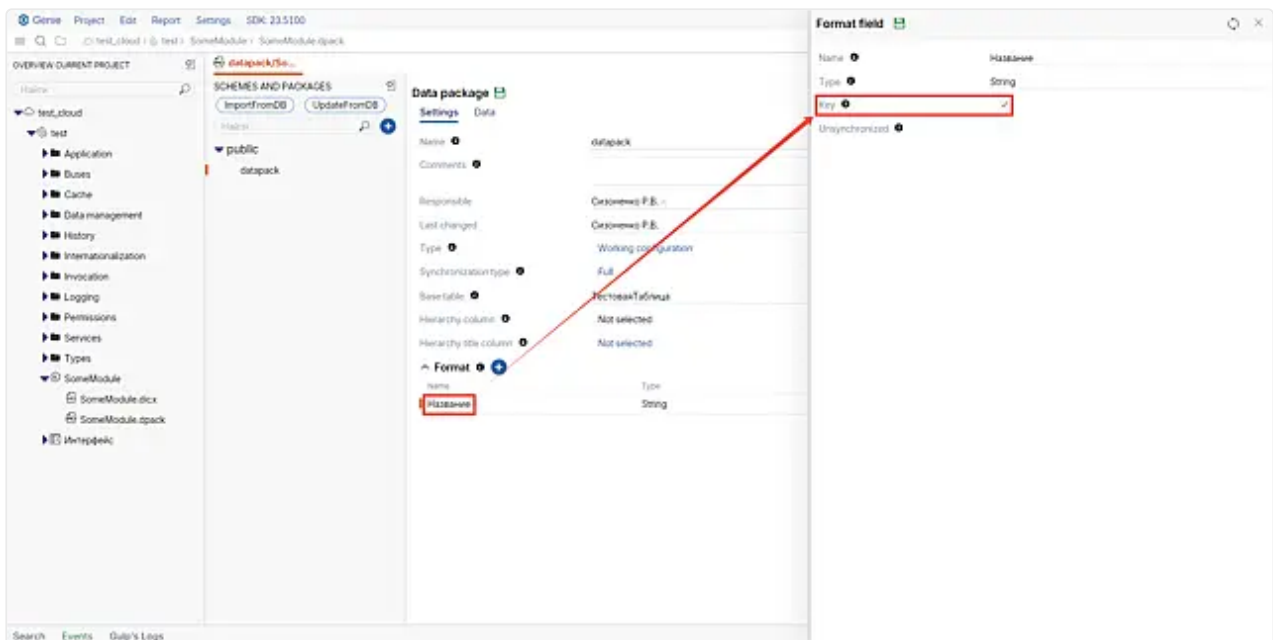


Таблица содержит колонку с именем Название и типом данных String.

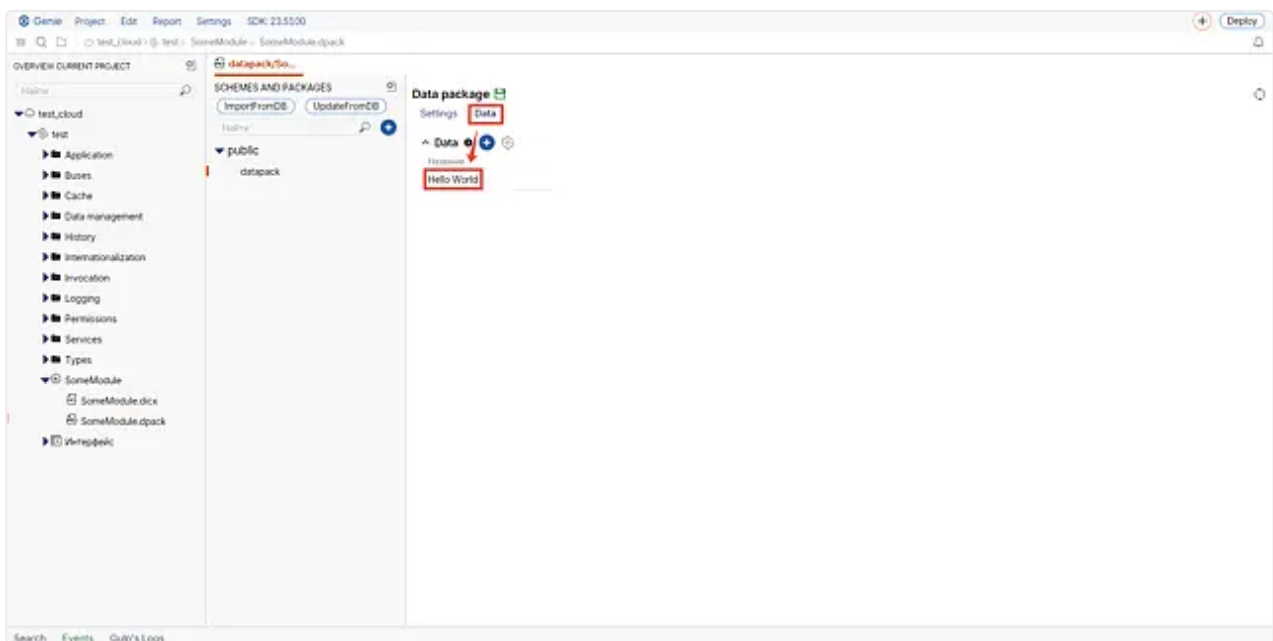
- [Пакет данных](#) SomeModule с пакетом datapack.



Тип синхронизации Full, а в атрибуте **Initialize format from table** указана таблица ТестоваяТаблица. В атрибуте **Format** для колонки Название указан флаг **Key**.



На вкладке пакета данных **Data** добавлено текстовое значение Hello World:



Файл test.entry имеет следующий вид:

```
1 {
2     "pytest_on": "test_cloud"
3 }
```

В каталоге **src** расположены файл service_test.py, в котором размещен исходный код теста:

```
1 import sbis_root as sbis
2 from unittest import TestCase
3
4 class TextFrameworkExample(TestCase):
5
6     def test_args(self):
7         db = sbis.GetConnectedDatabase()
8         res = sbis.SqlQuery( "SELECT \"Название\" FROM
9             \"ТестоваяТаблица\"")
10         self.assertEqual(res.Size(), 1)
```

и файл сборки теста CMakeLists.txt:

```
1 cmake_minimum_required(VERSION 3.5.2)
```

```
2 include("$ENV{SBISPlatformSDK_235100}/tools/cmake_support/sbis.cmake")
3 sbis_create_project("test-some-lib" test DEPENDS
4                     boost_system
5                     boost_filesystem
6                     boost_unit_test_framework)
7 sbis_add_sdk_dependencies( sbis-lib300 sbis-bl-core300 sbis-lang300
8 sbis-rpc300 sbis-xml300 sbis-xerces300 sbis-dbi300 sbis-dbi-postgres
9 sbis-dbi-sqlite sbis-bl-metadata-basic300 sbis-dbi-postgres sbis-db-
10 manager300)
11 target_sources(${target} PRIVATE
12 "main.cpp"
13 )
14 sbis_treat_warnings_as_errors()
15 sbis_add_ctest_integration()
16 sbis_enable_unity_build(20)
17 sbis_enable_static_build()
```